

CHAPTER 1

BASIC PYTHON PROGRAMMING

INTRODUCTION: Python is a programming language that is commonly used to create websites and applications, automate processes, and do data analysis. Python is a general-purpose programming language, which means it can be used for wide range of applications and isn't specialized for any particular problem. Due its flexibility and user-friendliness, python has become one of the most widely used programming language.

Python is a general purpose, dynamic, high-level and interpreted programming language. It supports Object Oriented Programming (OOPs) approach to develop applications. It is simple and easy to learn. It also consists of high-level data structures.

It is a versatile, powerful scripting language which makes it attractive for application development. The language's syntax and dynamic typing with interpreted nature, makes it an ideal language for scripting and rapid application development

APPLICATIONS OF PYTHON

- a. Web Applications
- b. Artificial Intelligence and Machine Learning
- c. GUI Application
- d. Software Development
- e. Business Application
- f. Audio and Video Construction
- g. 3D CADD Applications etc

It has 2 components for processing high level program language

- Interpreter
- Compiler

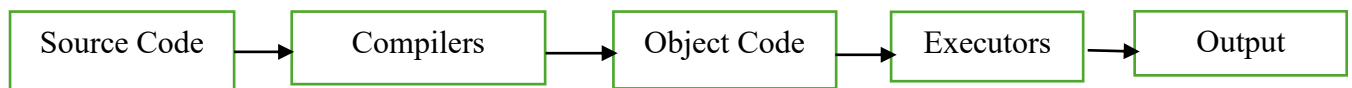
Interpreter: It reads the high-level program and executes it. It processes program little by little at a time reading line by line and performing computations. It stops the execution when it encounters an error.



Flowchart1: Program flow in Interpreter

Compiler: It reads the program; it translates it completely before the program starts running. If there are any errors, it gives the error list after the code is read completely.

The high-level program is called source code, the translated program is called object code or executable code.



Flowchart 2: Program execution in compiler

MODULES IN PYTHON

A python module is a file containing python definitions and statements. A module can define functions, classes and variables. A module can also include runnable code. Grouping related code into a module makes the code easier to understand and use. It also makes the code logically organised

Built in Python modules examples include

1. **math**
2. **os**
3. **sym**
4. **plt**

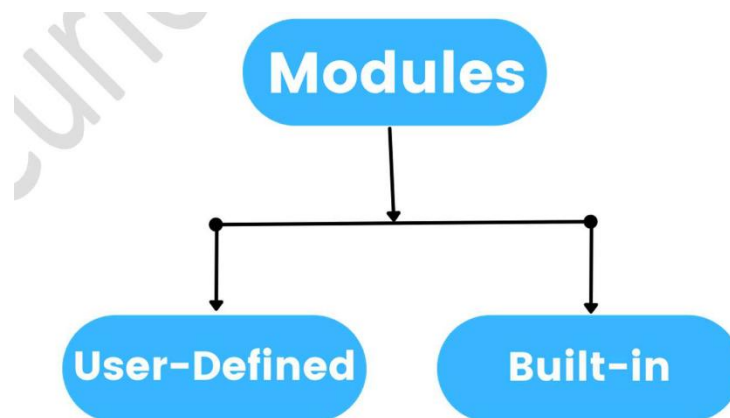


Figure 1: Module classification in Python

COMMENTS IN PYTHON

Python comment is an essential tool for the programmers. Comments are generally used to explain the code. We can easily understand the code if it has a proper explanation. A good programmer must use comments because in the future anyone who wants to modify the code as well as implement the new module can be done easily.

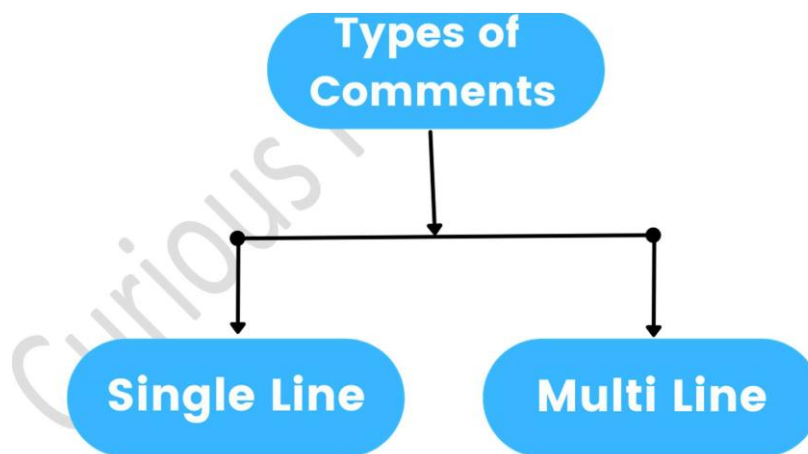


Figure 2: Comment classification in Python

SINGLE LINE COMMENT: Python single line comment starts with the hashtag symbol (#) with no white spaces and lasts till the end of the line. If the comment exceeds in one line, then put the hashtag in the next line and continue the comment.

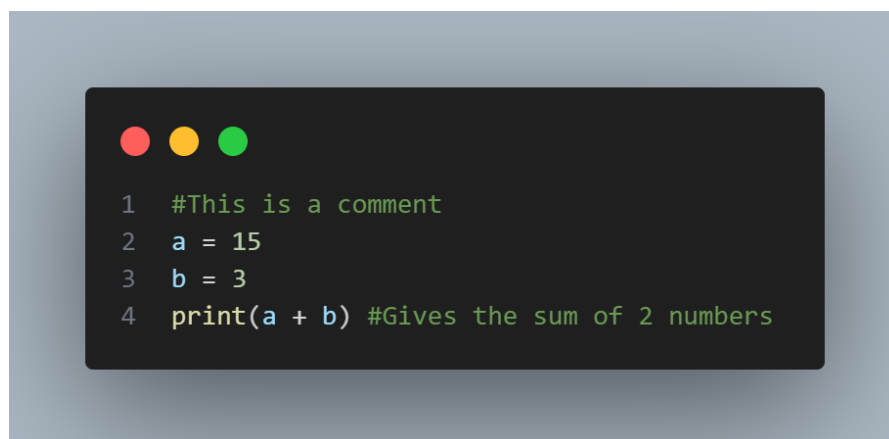


Figure 3: Single line comment in Python

MULTILINE COMMENTS: Python has no option to create multiline comment. However, there are different such ways through which we can write multiline comments. Python ignores string literals that are not assigned to a variable so we can use string literals as a comment.

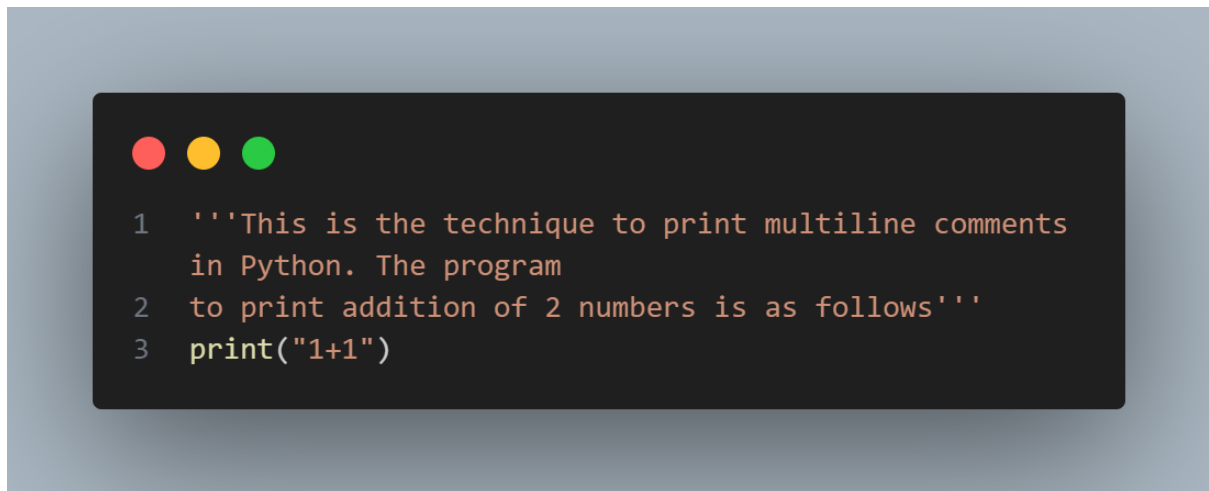


Figure 3: Multiline comment in Python

PIP IN PYTHON: PIP stands for **Preferred Installer Program**. It is a package-management system written in Python used to install and manage software packages. It connects to an online repository of public packages called Python Package Index.

PIP can also be configured to connect to other package repositories, provided that they comply to Python Enhancement Proposal 503

Example: `pip install matplotlib`

Faculty Incharge

CHAPTER 2

VARIABLES, DATATYPES AND KEYWORDS

VARIABLES: Variable is a name that is used to refer to memory location. Python variable is also known as an identifier used to hold value. In Python, there is no need to specify the type of variable because Python is a type infer language and smart enough to get variable type. **Variables can be a group of both letters and digits, but they have to begin with a letter or an underscore.**

NOTE on type infer language: Type inference is the automatic deduction of the data types of specific expressions in a programming language, usually done at compile time.

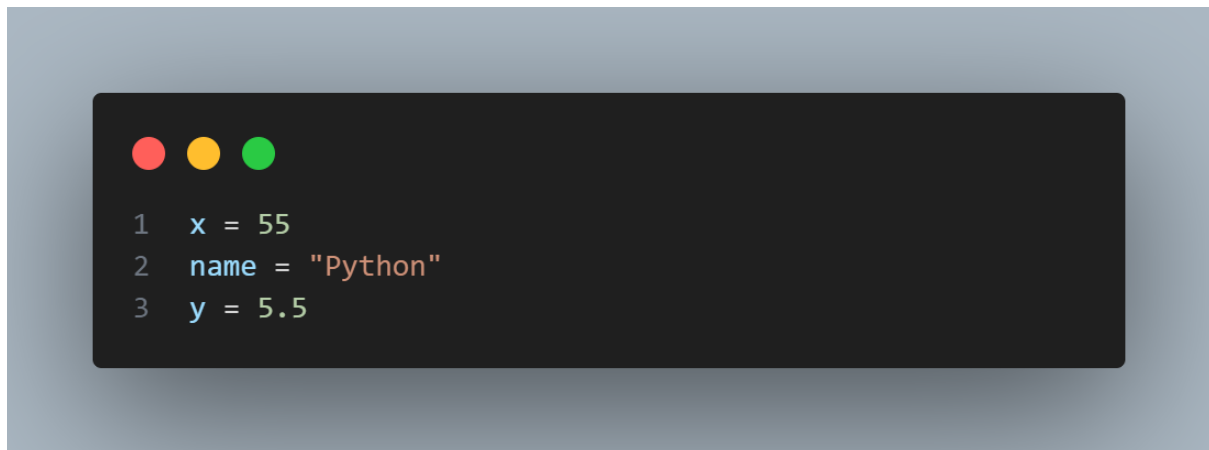


Figure 4: Variables in Python

IDENTIFIERS: Variables are examples of identifiers. An identifier is used to identify the literals used in the program. Some rules are to be followed.

RULES:

1. The first character of the variable should be an alphabet or underscore (_).
2. All characters except the first character may be an alphabet of lower-case (a-z), upper-case (A-Z), underscore or digit (0-9).
3. Identifier name should not contain any white-spaces or special character (! @, #, \$, %, ^, &, *)
4. Identifier name must not be similar to any keyword defined in the language.
5. Examples of valid identifiers: a_155, n998, etc.
6. Examples of invalid identifiers: 1a, n%4, n 9 etc.

NOTE: Variables can hold values, and every value has a data-type. Python is a dynamically typed language; hence we do not need to define the type of the variable while declaring it. The interpreter implicitly binds the value with its type.

Python enables us to check the type of the variable in the program using the `type()` function which returns the type of variable passed.

```
x = 55
name = "Python"
y = 5.5
print(type(x))
print(type(name))
print(type(y))

<class 'int'>
<class 'str'>
<class 'float'>
```

Figure 5: `type()` function in python

PYTHON DATA TYPES

1. **Numbers:** Numbers store numeric values. The integer, float and complex values belong to Python data type.

Python supports three types of numeric data

- a. **Integer:** Integer value can be any length such as integers 10, 8, 25, -188, -150 etc.
- b. **Float:** Float is used to store floating point numbers (Decimals) like 1.9, 9.908, 8.6658.
- c. **Complex number:** A complex number consists an ordered pair i.e. $x + iy$ where x denotes the real part and y denotes the imaginary part of the number.

2. Sequence Type:

- a. **String:** The string can be defined as the sequence of characters represented in quotation marks. In Python, we can use single, double and triple quotation marks to define a string.
- b. **List:** Python Lists are similar to arrays in C. However, the list can contain different data types. The items stored in the list are separated with comma (,) and enclosed in square brackets ([]).

- c. **Tuple:** A tuple is similar to the list in many ways. Like lists, tuples also contain the collection of items of different data types. The items of the tuple are separated with a comma (,) and enclosed in parenthesis ().
- 3. **Boolean:** Boolean type provides two built-in values, **True and False**. These values are used to determine the given statement true or false. It is denoted by class bool. True can be represented as a non-zero value or 'T' whereas false is represented by 0 or 'F'
- 4. **Set:** Python set is an unordered collection of data type. It is inerrable, mutable (can be modified after creation), and has unique elements. In set, the order of elements is undefined; it may return the changed sequence of the element.
- 5. **Dictionary:** Dictionary is an unordered set of key value pair items. It is like an associative array or hash table where each key stores a specific value. Key can hold any primitive data type, whereas value is an arbitrary Python object.

Faculty Incharge

CHAPTER 3

OPERATORS IN PYTHON

The operator can be defined as a symbol which is responsible for a particular operation between two operands. Python provides a variety of operators, which are described below

- Arithmetic operators
- Comparison operators
- Assignment operators
- Logical operators
- Bitwise operators
- Membership operators
- Identity operators

ARITHMETIC OPERATORS

These are the operators used with numeric values to perform common mathematical operations.

Operator	Name	Example
+	Addition	$x + y$
-	Subtraction	$x - y$
*	Multiplication	$x * y$
/	Division	x / y
%	Modulus	$x \% y$
**	Exponent	$x ** y$
//	Floor Division	$x // y$

Table 1: Arithmetic operators in Python

Basic arithmetic operations can be performed using these operators.

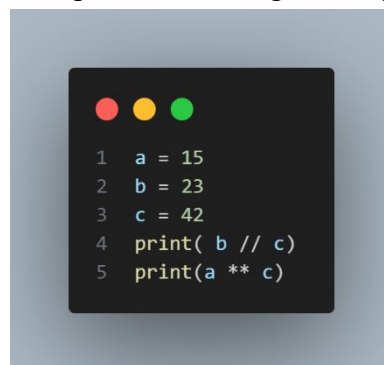


Figure 6: Arithmetic operator illustration

ASSIGNMENT OPERATORS

Assignment operators are used to assign values to the variables.

Operator	Example	Same as
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 8	x = x - 8
*=	x *= 6	x = x * 6
/=	x /= 5	x = x / 5
%=	x %= 4	x = x % 4
//=	x //= 1	x = x // 1
**=	x **= 2	x = x ** 2
&=	x &= 1	x = x & 1
=	x = 5	x = x 5
^=	x ^= 7	x = x ^ 7
>>=	x >>= 3	x = x >> 3
<<=	x <<= 8	x = x << 8

Table 2: Assignment operators in Python

COMPARISON OPERATORS

Comparison operators are used to compare two values.

Operator	Name	Example
==	Equal	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

Table 3: Comparison operators in Python

LOGICAL OPERATORS

Logical operators are used to combine conditional statements.

Operator	Description	Example
and	Returns true if both statements are true	$x < 5$ and $x < 10$
or	Returns true if one of the two statements is true	$x < 5$ or $x < 4$
not	Reverse the result, returns false if the result is true	not($x < 5$ and $x < 10$)

Table 3: Logical operators in Python

IDENTITY OPERATORS

Identity operators are used to compare the objects, not if they are equal, but they are the same object with the same memory location.

Operator	Description	Example
is	Returns true if both the values are the same object	x is y
is not	Returns true if both the values are not the same object	x is not y

Table 4: Identity operators in Python

MEMBERSHIP OPERATORS

These are used to test if a sequence is present in an object.

Operator	Description	Example
in	Returns true if the sequence with the specified value is present in the object	x in y
not in	Returns true if the sequence with the specified value is not present in the object	x not in y

Table 5: Membership operators in Python

BITWISE OPERATORS

Operator	Name	Description
&	AND	Sets each bit to 1 if both bits are 1
	OR	Sets each bit to 1 if one of the two bits are 1
^	XOR	Sets each bit to 1 if one of the two bits are 1
~	NOT	Inverts all the bits
<<	Zero fill left shift	Shift left by pushing zeros to the left
>>	Signed right shift	Shift right by pushing copies of left bit to right

Table 6: Bitwise operators in Python

Faculty Incharge

CHAPTER 4

LIST, TUPLES, SET, DICTIONARY

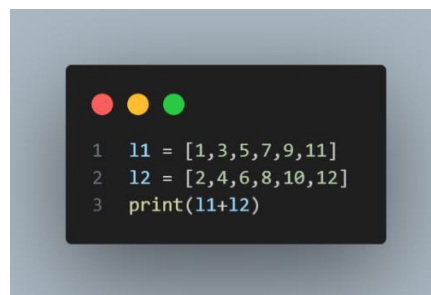
LIST:

Lists are dynamic sized arrays. Lists need not be homogenous always which makes it a most powerful tool in Python. A single list may contain data-types like integers, strings as well as objects. Lists are mutable; hence they can be altered even after their creation.

A list is defined as a collection of items or values of different types. The items in the list are separated by comma (,) and enclosed within square brackets [].

CHARACTERISTICS

1. Lists are ordered
2. The element of the list can be accessed by index
3. The lists are mutable
4. A list can store the number of various elements

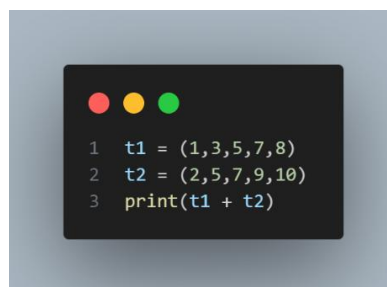


```
1 l1 = [1,3,5,7,9,11]
2 l2 = [2,4,6,8,10,12]
3 print(l1+l2)
```

Figure 7: List representation in Python

TUPLES

Python tuple is used to store the sequence of immutable Python objects. The tuple is similar to since the value of the items stored in the list can be changed whereas the tuple is immutable, and the value of the items stored in the tuple cannot be changed. **A tuple can be written as a collection of comma-separated (,) values enclosed with simple () brackets. The parenthesis is optional.**



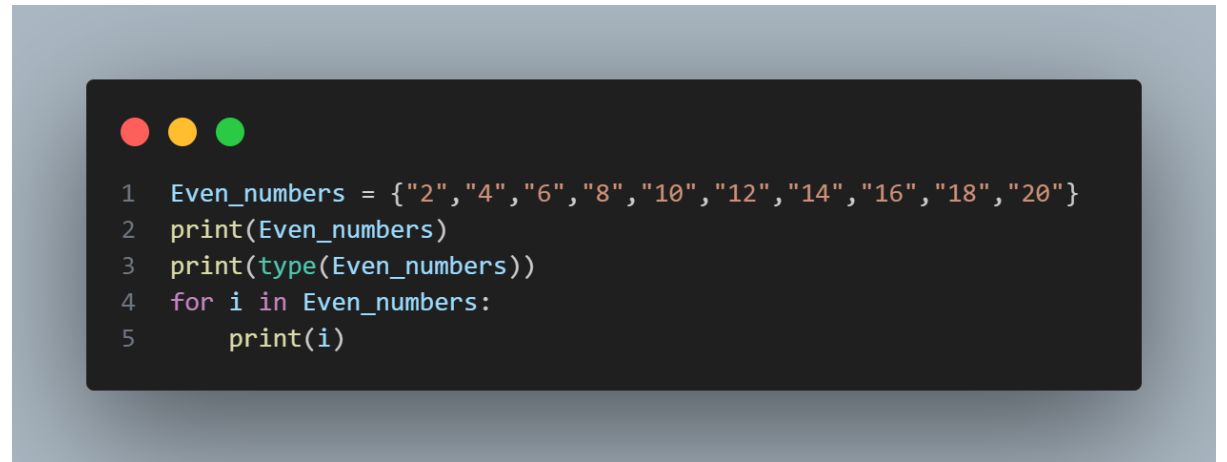
```
1 t1 = (1,3,5,7,8)
2 t2 = (2,5,7,9,10)
3 print(t1 + t2)
```

Figure 8: Tuple representation in Python

SET:

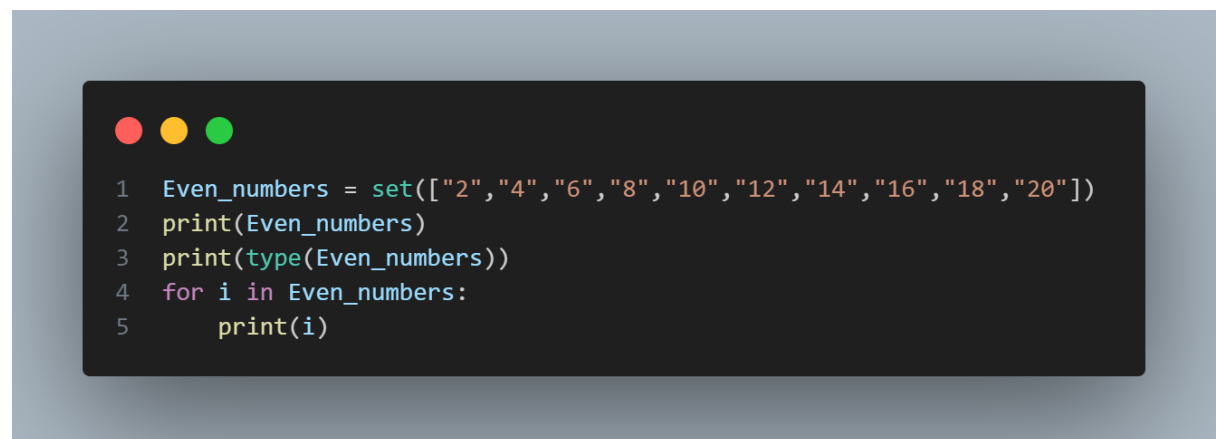
A set in Python is a collection of the unordered items. Each element in the set must be unique, immutable and the sets remove the duplicate elements. Sets are mutable which means we can modify it after its creation. **There is no index attached to the elements of the set i.e. we cannot directly access any element of the set by the index.** However, we can print them all together.

The set can be created by enclosing the comma-separated immutable items with the curly braces`{}`. Python also provides the `set()` method, which can be used to create the set by the passed sequence.



```
1 Even_numbers = {"2","4","6","8","10","12","14","16","18","20"}
2 print(Even_numbers)
3 print(type(Even_numbers))
4 for i in Even_numbers:
5     print(i)
```

Figure 9: Set representation in Python



```
1 Even_numbers = set(["2","4","6","8","10","12","14","16","18","20"])
2 print(Even_numbers)
3 print(type(Even_numbers))
4 for i in Even_numbers:
5     print(i)
```

Figure 10: Using the set() method

DICTIONARY:

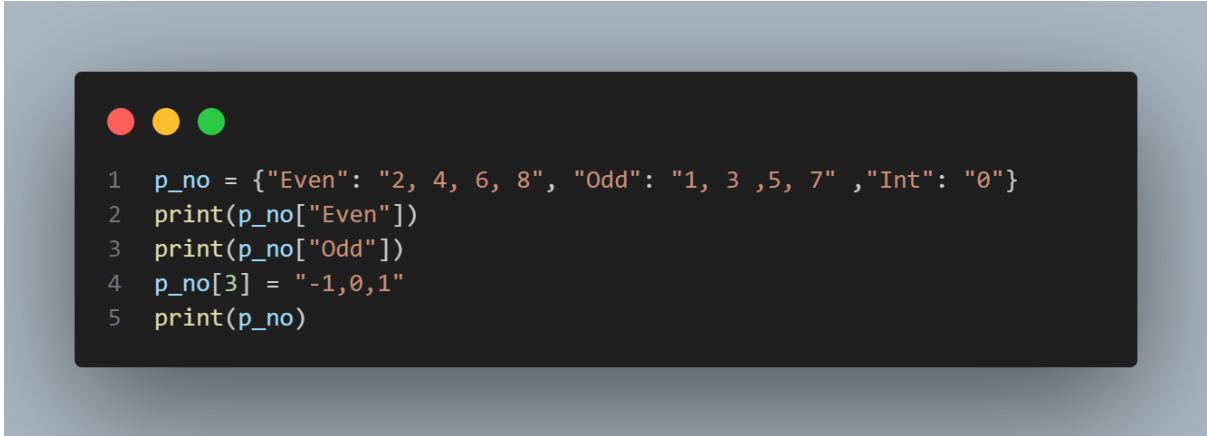
Python dictionary is used to store data in key-value pair format. These are unordered pairs. The dictionary in python can simulate the real-life data arrangement where some specific values exist for some particular key. It is the mutable data-structure. The dictionary is defined into elements keys and value.

- Keys must be a single element
- Value can be any type such as list, tuple, integer, etc.

The dictionary can be created by using multiple key-value pairs enclosed with the curly braces {} and each key is separated from its value by the colon (:).

The values can be accessed in the dictionary by using the keys as keys are unique in the dictionary.

The dictionary is mutable, it can be updated using the **update()** method.

A screenshot of a Python code editor with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The code is as follows:

```
1 p_no = {"Even": "2, 4, 6, 8", "Odd": "1, 3 ,5, 7" ,"Int": "0"}
2 print(p_no["Even"])
3 print(p_no["Odd"])
4 p_no[3] = "-1,0,1"
5 print(p_no)
```

Figure 11: Dictionary in Python

Faculty Incharge

CHAPTER 5

FLOW CONTROL STATEMENTS

In computer programming, control flow or flow of control is the order function calls, instructions and statements are executed or evaluated when a program is running. Many programming languages have what are called control flow statements which determine what the section of code is running in a program at any time. Decision making statements in programming languages decide the direction of flow of program execution.

Statements	Description
If Statement	The if statement is used to test a specific condition. If the condition is true, a block of code (if-block) will be executed.
If-else Statement	The if-else statement is similar to the if statement except for the fact that it also provides the block of code for the false case of the condition to be checked. If the condition provided in the statement is false, then the else statement will be executed.
Nested if Statement	Nested if statements enable us to use if statement inside an outer if statement.

Table 7: Table of control flow statement

1. If Statement

The if statement is used to test a particular condition and if the condition is true, it executes a block of code known as if-block. The condition of if statement can be any valid logical expression which can be either evaluated true or false.

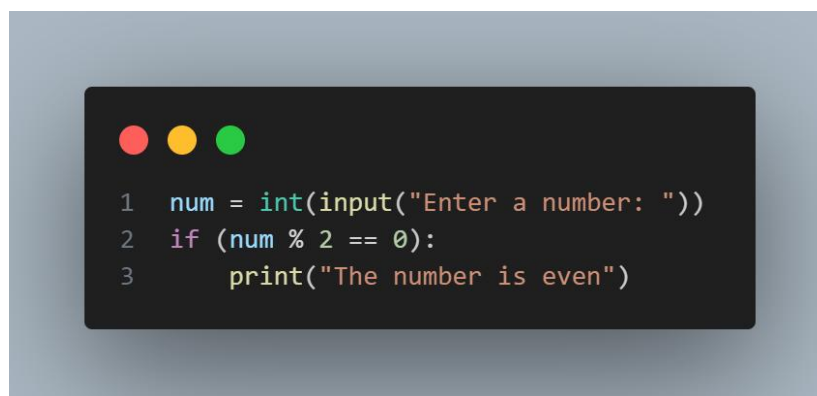


Figure 12: Execution in if loop

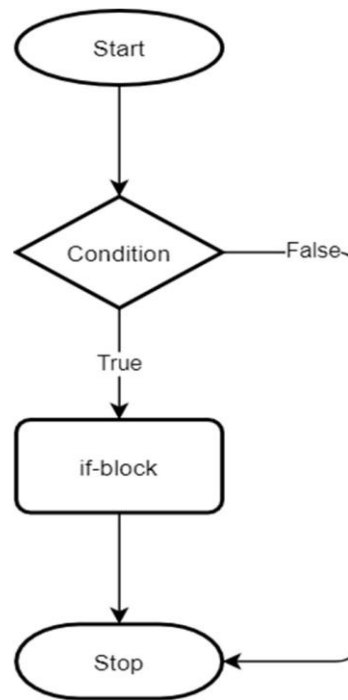


Figure 13: If statement flow chart

2. If-else Statement

The if-else statement provides an else block combined with the if statement which is executed in the false case of the condition. If the condition is true, if-block is executed. Otherwise, the else block is executed.

```
1 num = int(input("Enter a number: "))
2 if (num % 2 == 0):
3     print("The number is even")
4 else:
5     print("The number is odd")
```

Figure 14: Execution in else-if loop

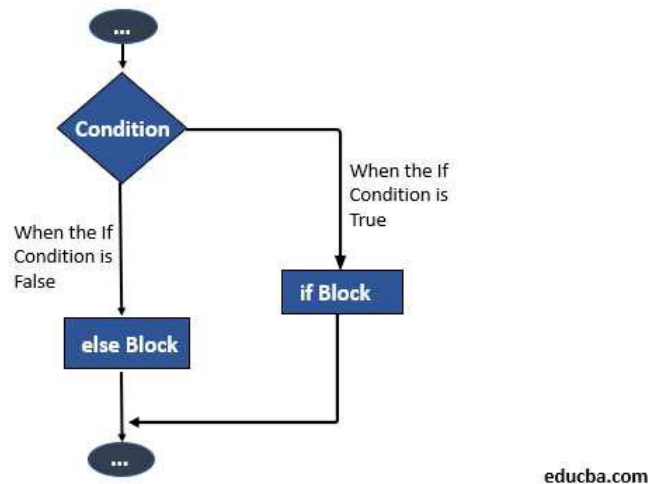


Figure 15: if-else statement flow chart

3. Elif Statement

The elif statement enables us to check multiple condition and execute the specific block of statements depending upon the true condition among the. We can have any number of elif statements depending upon the needs. However, using elif is optional. The elif statement works like an if-else-if ladder statement in C. It must be succeeded by an if statement.

```
1 number = int(input("Enter a number:"))
2 if number == 10:
3     print("The number is 10")
4 elif number == 50:
5     print("The number is 50")
6 elif number == 100:
7     print("The number is 100")
8 else:
9     print("The number is not equal to 10,50,100")
```

Figure 16: Execution of elif statement

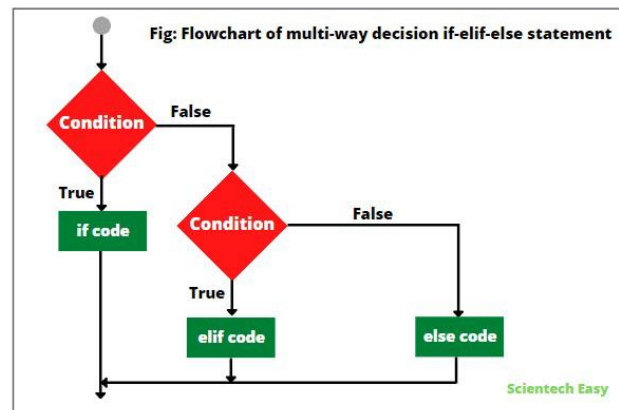


Figure 17: elif statement flow chart

4. Nested if Statement

A nested if statement is an if statement that is the target of another if statement. Nested if statements mean an if statement inside another if statement. Python allows us to nest if statements within if statements, i.e. we can place if statement inside another if statement.

```

1  m1 = int(input("Enter the marks in the first test: "))
2  m2 = int(input("Enter the marks in second test: "))
3  m3 = int(input("Enter the marks in third test: "))
4  if (m1 > m2):
5      if (m2 > m3):
6          total = m1 + m2
7      else:
8          total = m1 + m3
9  elif (m1 > m3):
10     total = m1 + m2
11 else:
12     total = m2 + m3
13 Avg = total / 2
14 print ("The average of the best two test marks is: ", Avg)

```

Figure 18: Execution of nested if statement



Figure 19: Nested if statement flow chart

Faculty Incharge

CHAPTER 6

LOOPS AND LOOP CONTROL STATEMENTS

Looping means repeating something over and over until a particular condition is satisfied. The following loops are available in Python to fulfil looping needs. Python offers 3 choices for running the loops. The basic functionality of all the techniques is the same, although the syntax and the amount of time required for checking the condition differs.

Name of the Loop	Description
While Loop	Repeats a statement or group of statements while a given condition is TRUE. It tests the condition before executing the loop body.
For Loop	This type of loop executes a code block multiple times and abbreviates the code that manages the loop variable
Nested Loops	We can iterate a loop inside another loop.

Table 8: Looping statements in Python

1. While Loop

The while loop is used to execute a block of statements continuously until a given condition is satisfied, and when the condition becomes false, the line immediately after the loop in the program is executed.



Figure 20: Execution in While loop

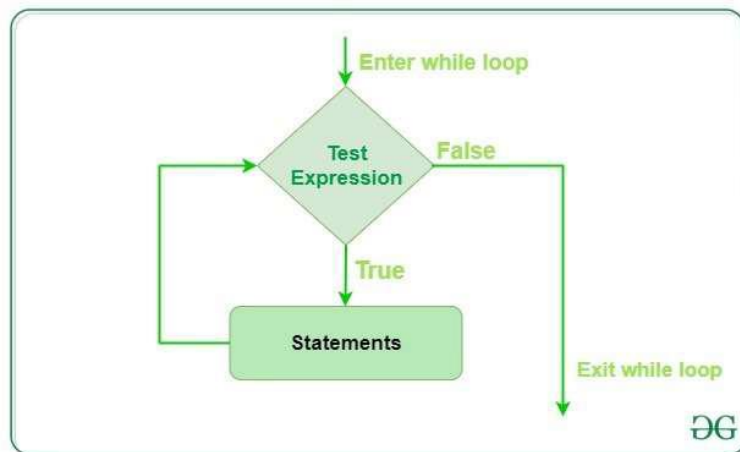


Figure 21: while loop execution flow chart

2. For Loop

The for loop in python is used for sequential traversal i.e. it is used for iterating over a sequence like string, tuple, set, list or dictionary. It is used to implement collection-based iterations.



Figure 22: Execution in for loop

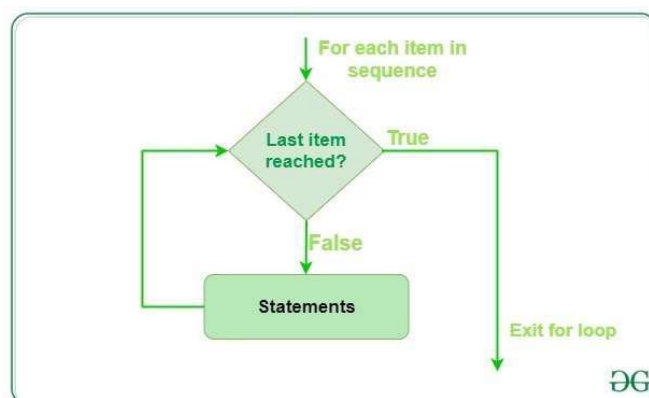
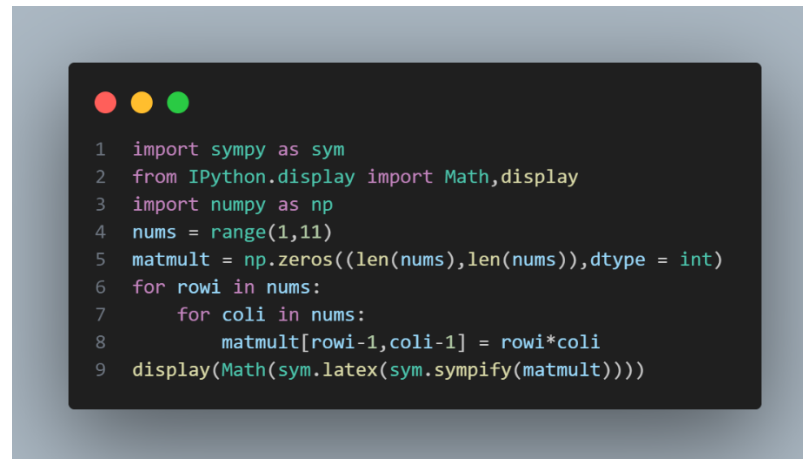


Figure 23: for loop execution flow chart

3. Nested Loops

In Python programming language there are two types of loops which are for loop and while loop. Using these loops, we can create nested loops in Python. Nested loops mean loops inside a loop. For example, while loop inside the for loop, for loop inside the for loop, etc.



```
1 import sympy as sym
2 from IPython.display import Math, display
3 import numpy as np
4 nums = range(1,11)
5 matmult = np.zeros((len(nums),len(nums)),dtype = int)
6 for rowi in nums:
7     for coli in nums:
8         matmult[rowi-1,coli-1] = rowi*coli
9 display(Math(sym.latex(sym.sympify(matmult))))
```

Figure 24: Execution in nested loops

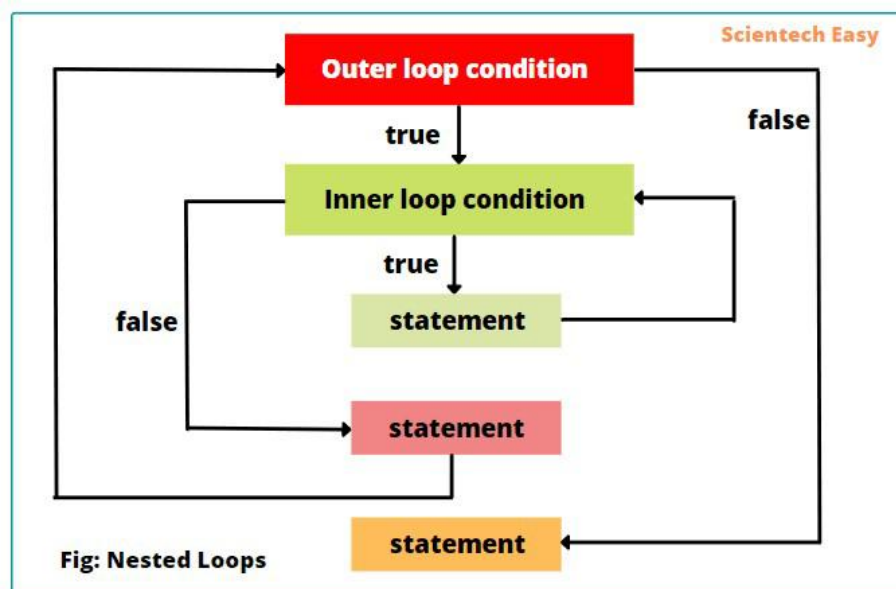


Figure 25: Nested loop execution flow chart

LOOP CONTROL STATEMENTS

Loop control statements change execution from their normal sequence. When execution leaves a scope, all automatic objects that were created in that scope are destroyed. Python supports the following control statements.

There are 3 loop control statements namely:

- a. **Continue**
- b. **Break**
- c. **Pass**

1. **Continue Statement**

It is a loop control statement that forces to execute the next iteration of the loop while skipping the rest of the code inside the loop for the current iteration only.

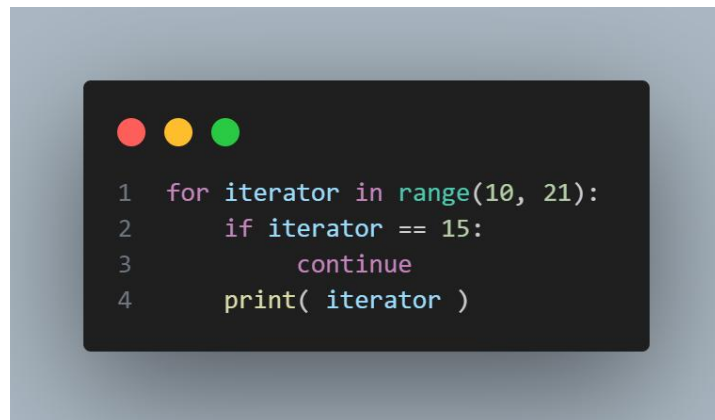


Figure 26: Continue statement

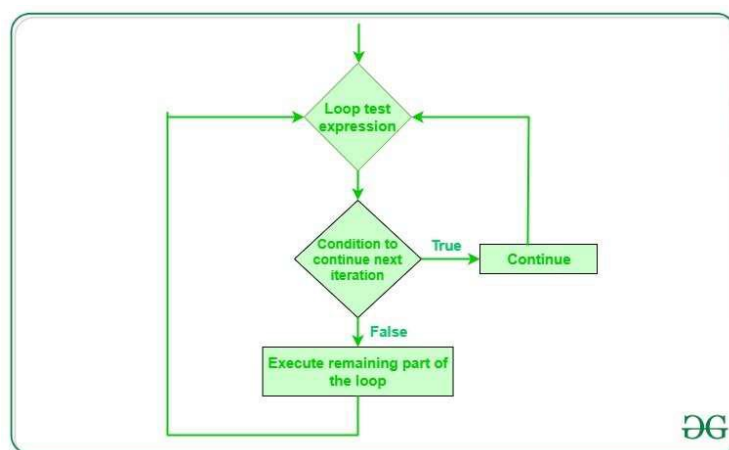
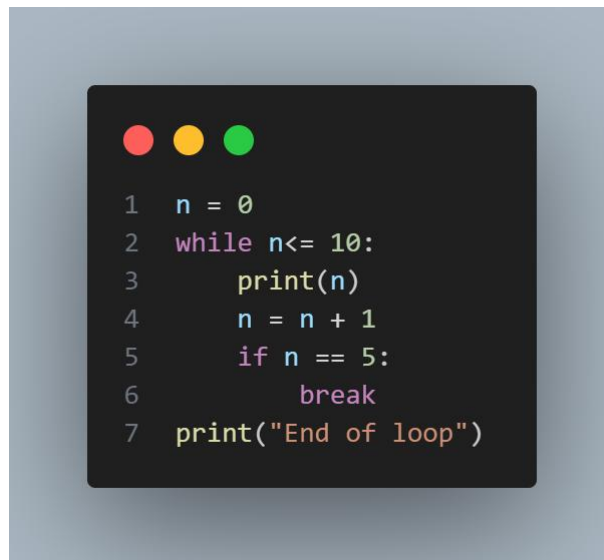


Figure 27: Continue execution flow chart

2. **Break Statement**

It brings the control out of the loop.



```
1 n = 0
2 while n <= 10:
3     print(n)
4     n = n + 1
5     if n == 5:
6         break
7 print("End of loop")
```

Figure 28: Break statement

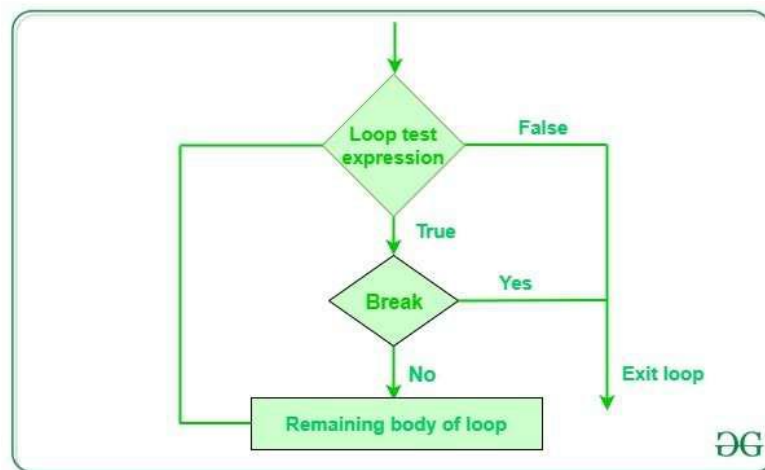
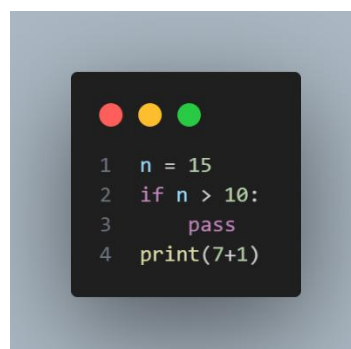


Figure 29: break execution flow chart

3. Pass Statement

We pass the arguments to write empty loops. Pass is also used for empty control statements, functions and classes



```
1 n = 15
2 if n > 10:
3     pass
4 print(7+1)
```

Figure 30: Pass statement

Faculty Incharge

CHAPTER 7

LIBRARIES IN PYTHON

Python libraries are a group of interconnected modules. It contains code bundles that can be reused in different programs and apps. The library contains built-in modules that provide access to system functionality such as file I/O that would otherwise be inaccessible to Python programmers, as well as modules written in Python that provide standardized solutions for many problems that occur in everyday programming. Most of the python libraries are usually written in C programming language. There are 5 Python libraries

1. NumPy
2. Pandas
3. SciPy
4. Matplotlib
5. Scikit-learn
6. Sympy

NumPy: Numerical python is a machine learning library. It can handle big matrix and multi-dimensional data. These are initialised by using the command *import numpy as np*

Pandas: Pandas consists of variety of analysis of tools and configurable and high-level data structure. The pandas are initialised by using the command *import pandas as pd*

SciPy: Scientific python is an open source of high level scientific, computational package. It is based on NumPy extension. As it is a NumPy extension once NumPy is initiated, then we use the function/command *import scipy as sp*

Matplotlib: It is also open-source library that plots high-definition figures such as pie charts, histograms etc. The matplotlib is initialised by the command *import matplotlib.pyplot as plt*

Scikit-learn: The library contains a lot of efficient tools for machine learning and statistical modelling including classification, regression, clustering and dimensional reduction. We import the data sets using the command *from sklearn.datasets import*

Sympy: Sympy is a python library for symbolic mathematics, It aims in providing complex mathematical functions to compute mathematics. Importing command *import sympy as sym.*

Faculty Incharge

CHAPTER 8

FUNCTIONS IN PYTHON

Functions are the most important aspect of an application. A function can be defined as the organised block of reusable code, which can be called whenever required. A function can be defined as the organised block of reusable code, which can be called whenever required. Python allows us to divide a large program into basic building blocks known as a function. The function contains a set of programming statements enclosed by {}. A function can be called multiple times to provide reusability and modularity to the Python program.

The function helps the programmer to break the program into smaller parts. It organises the code very efficiently and avoids repetition of code. As the program grows, function makes the program more organised. Python provides us various built-in functions like **range()** or **print()**. Although, the user can create functions which are called as user-defined functions. There are mainly two types of functions

- a. **User defined functions:** The user defined functions are those functions that are defined by the user to perform specific task
- b. **Built-in functions:** The built in functions are those functions that are pre-defined in python.

ADVANTAGES OF FUNCTIONS

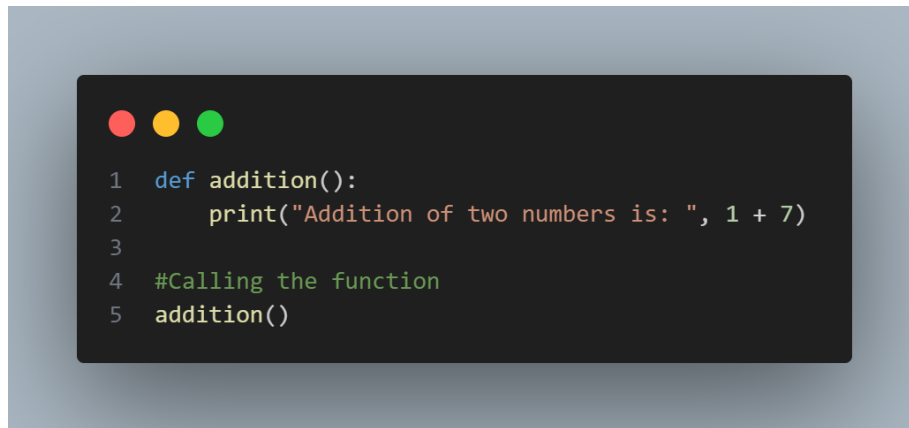
There are the following advantages of Python functions.

- Using functions, we can avoid rewriting the same logic/code again and again in a program.
- We can call Python functions multiple times in a program and anywhere in a program.
- We can track a large Python program easily when it is divided into multiple functions.
- Reusability is the main achievement of Python functions.
- However, Function calling is always overhead in a Python program.

CREATING A FUNCTION

- The def keyword, along with the function name is used to define the function.
- The identifier rule must follow the function name.
- A function accepts the parameter (argument) and they can be optional
- The function block is started with a colon (:)

- Block statements must be at the same indentation.
- The return statement is used to return a value. A function can have only one return.

A code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The code is as follows:

```
1 def addition():
2     print("Addition of two numbers is: ", 1 + 7)
3
4 #Calling the function
5 addition()
```

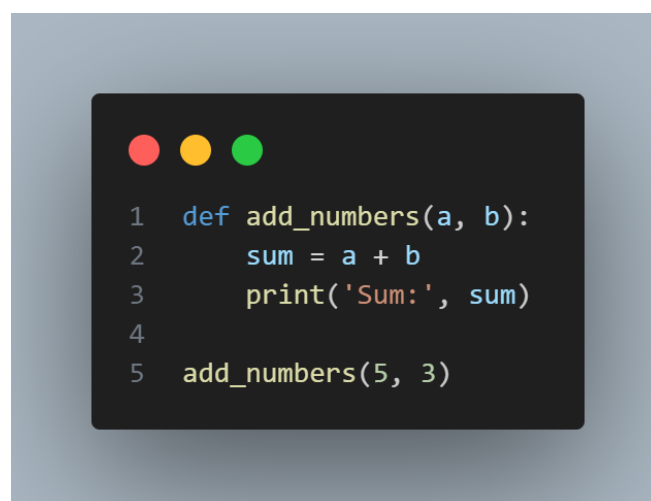
Figure 31: Illustration of a function

FUNCTION CALL

In Python, after the function is created, we can call it from another function. A function must be defined before it is called/function call; otherwise, the python interpreter gives an error. To call the function, use the function name followed by parenthesis ().

ARGUMENTS IN FUNCTION

The arguments are types of information which can be passed into the function. The arguments are specified in the parentheses. We can pass any number of arguments, but they must be separated with comma.

A code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The code is as follows:

```
1 def add_numbers(a, b):
2     sum = a + b
3     print('Sum:', sum)
4
5 add_numbers(5, 3)
```

Figure 32: Function Arguments

TYPES OF ARGUMENTS

- a. Required Arguments
- b. Keyword Arguments
- c. Default Arguments
- d. Variable-Length Arguments

THE range() FUNCTION

Range generates a sequence of numbers from 0 to the ending number by incrementing by 1 and without including the last number/ending number.

range(a): Generates a sequence of numbers from 0 to a, excluding a, incrementing by 1.

range(a, b): Generates a sequence of numbers from a to b excluding b, incrementing by 1.

range(a, b, c): Generates a sequence of numbers from a to b excluding b, incrementing by c.

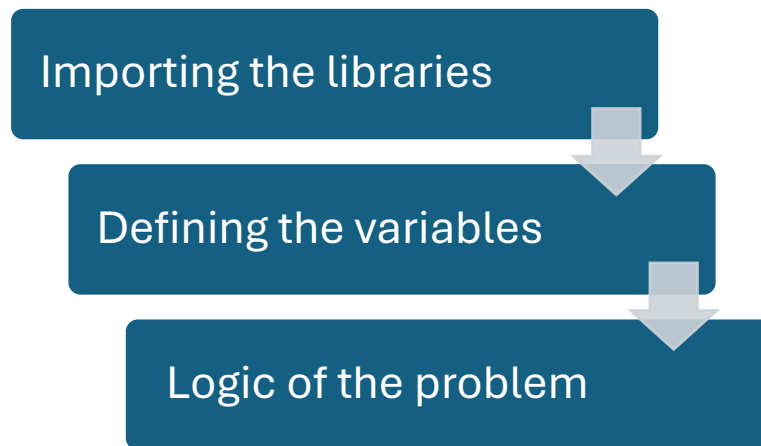
Faculty Incharge

CHAPTER 9

FLOWCHART FOR PYTHON PROGRAMMING

The Python programming gives a scope to solve mathematics with it. There are several libraries that are defined in Python to solve mathematics.

The flow of the program is as follows



Flowchart 3: Python Programming

Faculty Incharge